A SEMINAR REPORT ON

" Clinic Management System with Disease Prediction and Appointment Booking "

SUBMITTED TO SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE

MASTER OF TECHNOLOGY IN COMPUTER ENGINEERING

BY

Candidate Name: Saurabh Mapari Exam No:M240501015

Under the guidance of

Prof R.S.Paswan

DEPARTMENT OF COMPUTER ENGINEERING

Society for Computer Technology and Research's

PUNE INSTITUTE OF COMPUTER TECHNOLOGY

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

2025-2026

DEPARTMENT OF COMPUTER ENGINEERING

Society for Computer Technology and Research's

PUNE INSTITUTE OF COMPUTER TECHNOLOGY

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

PUNE-411043

CERTIFICATE

This is to certify that the seminar report entitled

"Clinic Management System with Disease Prediction and Appointment Booking"

Submitted by

Name of the Candidate: Saurabh Mapari Exam No: M240501015

is a bonafide work carried out by them under the supervision of Dr. Sheetal Sonawane and it is submitted towards the partial fulfilment of the requirement of SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY for the award of the degree of Master of Technology in Computer Engineering

Prof R.S.Paswan	Dr. B, A. Sonkamble
Internal Guide	Head
Dept of Computer Engg	Dept of Computer Engineering
PICT, Pune	PICT, Pune

Internal Examiner External Examiner

Acknowledgement

I take this opportunity to express my profound gratitude and deep regards to my esteemed internal guide **Prof R.S.Paswan** for her exemplary guidance, continuous encouragement, and unwavering support throughout the course of this seminar. Her insightful suggestions, timely feedback, and invaluable expertise have been instrumental in shaping the direction and quality of this work. I am sincerely thankful for the time and effort she dedicated to mentoring me, despite her many commitments.

I would also like to extend my heartfelt thanks to **Dr. B, A. Sonkamble**, Head of the Department of Computer Engineering, **SCTR's Pune Institute of Computer Technology (PICT)**, for providing the necessary infrastructure, resources, and an intellectually stimulating environment that significantly contributed to the successful completion of this seminar.

My sincere appreciation also goes to all the faculty members, technical staff, and administrative personnel of the Computer Engineering Department at PICT for their constant support and cooperation during the seminar period.

I am immensely grateful to my **family** for their unconditional love, moral support, and patience, which have been a source of strength and inspiration throughout this endeavor. I also wish to thank my **friends and peers**, who have provided encouragement, shared insightful discussions, and helped me stay motivated during challenging times.

This seminar journey has been a valuable learning experience, and I am truly thankful to everyone who has been a part of it in one way or another.

Index

List of Tables

List of Figures

Abstract:

Healthcare institutions often face challenges in managing patient records, appointment scheduling, and providing timely medical guidance. Traditional clinic management systems primarily focus on administrative tasks and often lack intelligent decision-support capabilities for patients. To address these limitations, an AI-Powered Clinic Management System with Disease Prediction and Appointment Booking has been developed. The system integrates Machine Learning techniques with modern web technologies to provide an intelligent, efficient, and user-friendly healthcare management platform.

The application enables users to register, log in securely, predict diseases based on symptoms, receive doctor recommendations, and book appointments online. A Naïve Bayes Machine Learning algorithm is employed for disease prediction, utilizing symptom-based analysis to identify the most probable disease and recommend the appropriate medical specialist. The system also provides role-based access control for patients and administrators, ensuring secure management of healthcare services and data.

The application is developed using Java, Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, and MySQL. The backend architecture follows a layered design consisting of Controller, Service, Repository, and Database layers, ensuring modularity, maintainability, and scalability. The system efficiently manages doctor information, appointment schedules, patient interactions, and disease prediction processes through an integrated web-based interface.

To support real-world deployment, the application is containerized using Docker and deployed on the Render cloud platform, while Railway MySQL is used as the cloud-hosted database service. GitHub is utilized for source code management, version control, and deployment integration. These technologies provide portability, scalability, and ease of maintenance, enabling the application to be accessed from any location through a web browser.

The integration of Machine Learning with healthcare management functionalities enables intelligent decision support, helping patients identify potential diseases and consult appropriate specialists more efficiently. By automating disease prediction and appointment management, the system reduces manual effort, improves healthcare accessibility, enhances operational efficiency, and provides a better user experience for both patients and healthcare providers.

The developed system demonstrates the successful integration of Machine Learning, web development, database management, containerization, and cloud deployment technologies in a healthcare environment. Furthermore, the project serves as a foundation for future enhancements such as telemedicine support, electronic health records, real-time analytics, and advanced AI-driven healthcare recommendation systems.

Keywords: Artificial Intelligence, Machine Learning, Naïve Bayes, Disease Prediction, Clinic Management System, Appointment Booking, Spring Boot, MySQL, Docker, Cloud Deployment, Render, Railway, Healthcare Management.

CHAPTER 1

INTRODUCTION

1.1 Introduction:

The healthcare industry is rapidly evolving with the integration of digital technologies and intelligent systems. Traditional clinic management systems mainly rely on manual processes such as paper-based appointment booking, handwritten patient records, and offline doctor scheduling. These conventional approaches are time-consuming, error-prone, and inefficient for handling a large number of patients.

To overcome these limitations, the proposed project is developed as a web-based healthcare management application integrated with Machine Learning techniques. The system automates clinic operations and assists patients in preliminary disease prediction based on selected symptoms.

The proposed system integrates multiple healthcare functionalities into a single platform including:

- Disease prediction using Machine Learning
- Doctor recommendation
- Online appointment booking
- Doctor slot management
- User and hospital management

The system supports three major user roles:

- Patient
- Hospital/Doctor
- Admin

The project is developed using modern technologies such as Spring Boot, MySQL, JPA, Hibernate, HTML, CSS, Bootstrap, and Thymeleaf. A Naïve Bayes Machine Learning model is used for symptom-based disease prediction.

The main goal of the system is to provide a smart, secure, and efficient healthcare solution that improves patient experience and simplifies hospital management.

1.2 Motivation:

Many clinics and hospitals still use traditional methods for appointment booking and patient record management. These systems often create problems such as:

- Long waiting times
- Appointment conflicts
- Poor management of doctor schedules
- Manual errors in patient records
- Lack of intelligent disease prediction support

Patients frequently face difficulties in finding suitable doctors and booking appointments. Moreover, there is no integration between disease diagnosis and appointment systems in many existing healthcare applications.

Machine Learning has shown significant potential in healthcare applications, especially in disease prediction and diagnosis support systems. By integrating Machine Learning with healthcare management, clinics can provide faster and smarter services.

The motivation behind this project is to:

- Automate clinic operations
- Improve healthcare accessibility
- Reduce manual work and human errors
- Provide preliminary disease prediction
- Offer a user-friendly healthcare platform

CHAPTER 2

LITERATURESURVEY

No.	Paper	Methodologies / Tools Used	Dataset	Key Findings	Limitations	Applications
1	Symptom-Based Disease Prediction using Machine Learning Techniques (2023)	Naïve Bayes, Decision Tree, Random Forest	UCI Symptom -Disease Dataset	Naïve Bayes performed well for categorical symptom features with low computation time	Assumes conditional independence between symptoms	Primary disease screening systems
2	Comparative Analysis of Machine Learning Algorithms for Healthcare Prediction (2024)	SVM, KNN, Logistic Regression	Kaggle Medical Dataset	SVM achieved high classification accuracy in multi-class disease prediction	High training time and parameter tuning complexity	Clinical decision support systems
3	Design and Implementation of Web-Based Hospital Management System (2022)	Java EE, MySQL, MVC Architecture	Real-time hospital records	Improved operational efficiency and centralized data handling	No intelligent disease prediction module	Hospital administratio n automation
4	Probabilistic Model for Medical Diagnosis using Gaussian Naïve Bayes (2023)	Gaussian Naïve Bayes	Multi-symptom dataset	Effective probabilistic classification with fast prediction speed	Lower performance for correlated features	Symptom-based diagnostic tools
5	Intelligent Appointment Scheduling System using Spring Boot Framework (2024)	Spring Boot, Hibernate, REST APIs	Simulate d patient dataset	Reduced scheduling conflicts using structured time-slot allocation	No predictive analytics integration	Online appointment platforms

6	Disease Classification using Ensemble Machine Learning Methods (2023)	Random Forest, Gradient Boosting	UCI Healthcare Dataset	Ensemble models achieved highest accuracy compared to single classifiers	Increased computational cost	Advanced disease classification
7	Predictive Analytics in Healthcare using Statistical Learning Models (2022)	Logistic Regression, Naïve Bayes	Public health dataset	Logistic regression suitable for binary classification tasks	Limited scalability for multi-class problems	Health risk prediction
8	Deep Learning Approaches for Medical Diagnosis and Disease Detection (2024)	ANN, CNN	Large-scale hospital dataset	Deep learning models achieved superior accuracy with large data	Requires high computational resources and large datasets	Complex disease detection
9	Role-Based Secure Healthcare Information Systems using Spring Security (2023)	Spring Security, RBAC Model	Simulated clinical dataset	Role-based authentication improved system security and data isolation	Complexity in access control configuration	Secure hospital information systems
10	AI-Based Healthcare Recommendation Systems using Naïve Bayes and SVM (2025)	Naïve Bayes, SVM	Multi-symptom structured dataset	Naïve Bayes efficient for high-dimensional categorical data	Performance depends heavily on dataset quality	Doctor recommendation systems

Key Findings

1. Machine Learning algorithms such as Naïve Bayes, Random Forest, and SVM are highly effective for symptom-based disease prediction systems.
2. Naïve Bayes provides fast prediction speed and better performance for categorical healthcare datasets with lower computational complexity.
3. Web-based healthcare management systems improve appointment scheduling, doctor management, and overall operational efficiency.
4. Role-based authentication and secure healthcare architectures improve data security and controlled access in hospital management systems.

Research Gaps

1. Most existing healthcare systems do not integrate disease prediction with appointment booking in a single platform.
2. Many hospital management systems lack intelligent doctor recommendation and predictive analytics modules.
3. Existing Machine Learning models often require large real-time medical datasets for higher accuracy and scalability.
4. Several systems focus only on prediction accuracy but provide limited support for secure role-based healthcare management and user interaction.

CHAPTER 3

PROBLEM DEFINITION, OBJECTIVES, AND SCOPE

3.1 Problem Definition

Many clinics and healthcare centers still rely on traditional manual methods for managing appointments, doctor schedules, and patient records. These conventional systems are inefficient and often lead to delays, errors, and poor management of healthcare services. Patients frequently face difficulties in booking appointments, checking doctor availability, and maintaining medical records properly.

Manual clinic management systems create several problems such as:

- Delays in appointment booking
- Poor management of doctor time slots
- Difficulty in maintaining patient records
- Increased chances of human errors
- Lack of intelligent disease prediction support
- Limited accessibility to healthcare services

In many existing systems, there is no integration between disease diagnosis and appointment booking. Patients must first consult doctors manually before understanding possible health conditions. This process increases waiting time and reduces healthcare efficiency.

Therefore, there is a need for a smart and secure healthcare management system that can automate clinic operations and provide preliminary disease prediction using Machine Learning techniques.

The proposed Clinic Management System with Disease Prediction and Appointment Booking is designed to address these issues by integrating:

- Disease prediction
- Doctor recommendation
- Online appointment booking
- Slot management
- Secure user management

The system aims to improve healthcare efficiency, reduce manual effort, and provide a better experience for both patients and hospitals.

3.2 Objectives

The main objectives of the proposed system are:

1. To develop a web-based Clinic Management System.
2. To provide role-based access for:
    - Patient
    - Hospital/Doctor
    - Admin
3. To allow patients to predict diseases based on selected symptoms using the Naïve Bayes Machine Learning algorithm.
4. To recommend appropriate doctors according to the predicted disease.
5. To provide online appointment booking functionality.
6. To help hospitals manage doctors and time slots efficiently.

3.3 Scope

The proposed system is designed for clinics, hospitals, and healthcare centers to automate healthcare operations and provide intelligent disease prediction support.

Patient Scope

The patient module provides:

- User registration and login
- Symptom selection
- Disease prediction
- Doctor recommendation
- Appointment booking
- Appointment history management

Hospital Scope

The hospital module provides:

- Doctor management
- Time slot management
- Appointment handling
- Patient monitoring

- Doctor availability management

Admin Scope

The admin module provides:

- User management
- Hospital management
- Doctor approval and monitoring
- System management and control

Machine Learning Scope

The Machine Learning module provides:

- Symptom-based disease prediction
- Multi-class disease classification
- Comparative analysis of algorithms
- Selection of best-performing model

CHAPTER 4

METHODOLOGY

4.1 System Architecture

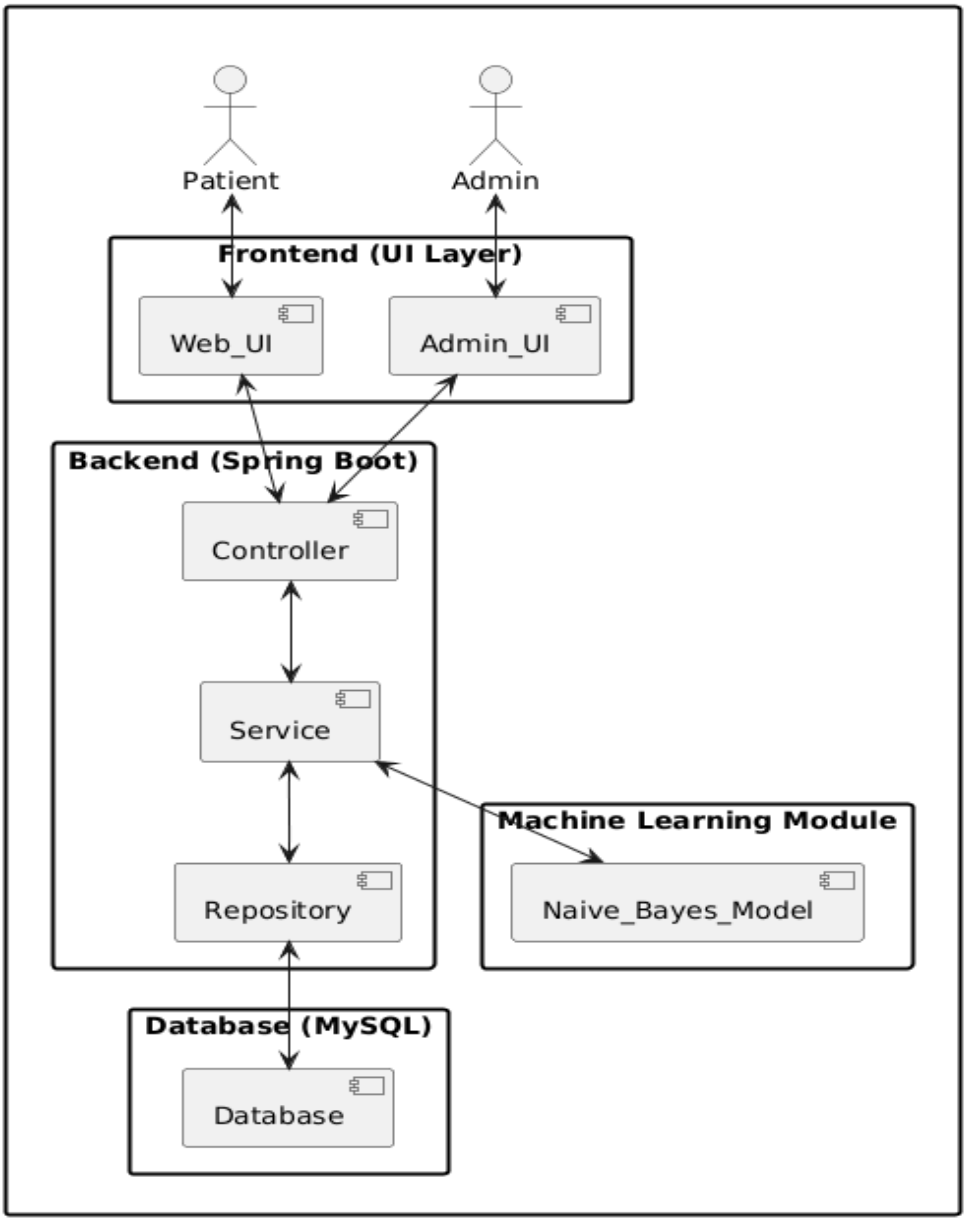


Figure 4.1: System Architecture Diagram

The system architecture of the proposed Clinic Management System consists of four major components: Frontend Layer, Backend Layer, Machine Learning Module, and Database Layer. The frontend provides interfaces for patients and administrators, while the Spring Boot backend handles business logic through Controller, Service, and Repository layers.

4.2 System Workflow

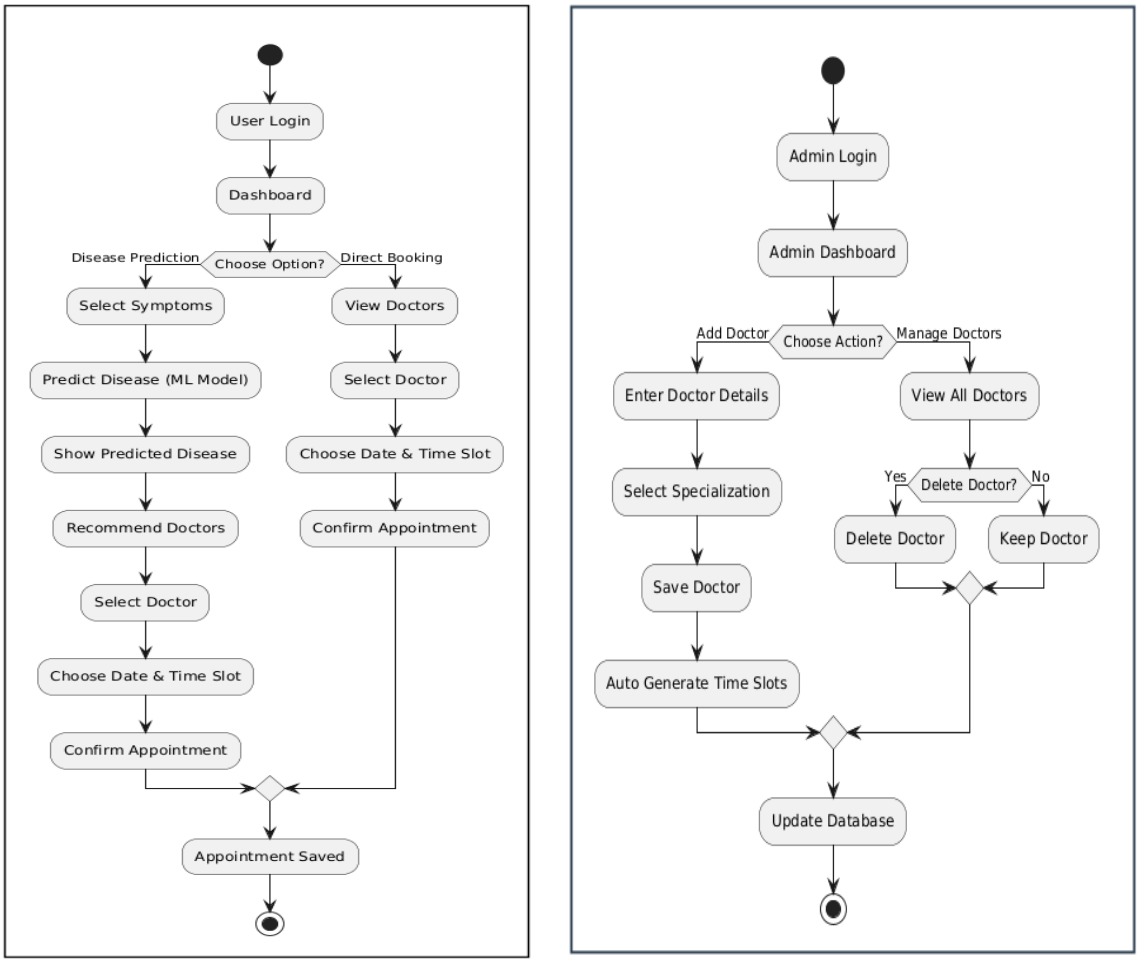


Figure 4.2: System Workflow Diagram

The system workflow illustrates the complete operational flow for both patient and admin modules. It shows how patients can perform disease prediction and appointment booking, while administrators manage doctors, specializations, and time slots through the admin dashboard.

The workflow ensures smooth interaction between users, Machine Learning prediction, appointment management, and database operations for efficient healthcare service management.

4.3 Module-Wise Explanation

The proposed Clinic Management System is divided into three major modules namely Patient Module, Hospital Module, and Admin Module. Each module is designed to perform specific functionalities for efficient healthcare management and smooth system operation.

Patient Module

The Patient Module provides functionalities related to patient activities within the system. It allows users to register and log in securely to access healthcare services.

The major features of the patient module include:

- User registration and login
- Symptom selection
- Disease prediction using Machine Learning
- Doctor recommendation based on predicted disease
- Online appointment booking
- Viewing appointment history

This module helps patients obtain preliminary disease information and book appointments easily without visiting hospitals physically for initial consultation.

Hospital Module

The Hospital Module is responsible for managing doctors, appointments, and available time slots. Hospitals can efficiently organize healthcare services using this module.

The major features of the hospital module include:

- Doctor management
- Time slot management
- Appointment handling
- Patient monitoring

This module improves doctor availability management and helps hospitals maintain efficient appointment scheduling.

Admin Module

The Admin Module controls overall system management and monitoring. The admin manages users, hospitals, and system-related activities.

The major features of the admin module include:

- User management
- Hospital management
- Doctor approval
- Monitoring system activities

The admin module ensures secure and proper functioning of the application through centralized system control.

4.4 Dataset Description

The proposed system uses a structured symptom-based medical dataset for disease prediction and doctor recommendation. The dataset is designed for multi-class disease classification and contains information related to diseases, symptoms, and doctor specializations.

Dataset Type

The dataset used in the system is:

- Structured dataset
- Symptom-based dataset
- Multi-class classification dataset

The dataset is organized in tabular format and is suitable for Machine Learning classification algorithms.

Key Characteristics

The dataset contains:

- Categorical symptom features
- Multiple disease classes
- Doctor specialization mapping

Each disease in the dataset is associated with:

- Five symptoms
- One doctor specialization

The dataset supports both:

- Disease prediction
- Doctor recommendation

Dataset Structure

Column Name	Description
Disease	Target variable (Predicted output)
Specialization	Type of doctor required
Symptom1–Symptom5	Symptoms associated with the disease

Table 4.1: Dataset Structure

Data Source

The dataset was manually prepared using various healthcare references and medical resources. The major data sources include:

- WHO guidelines
- Medical references
- Online healthcare sources

The dataset was designed specifically for symptom-based disease prediction and healthcare recommendation purposes.

4.5 Data Preprocessing

Data preprocessing is an important step in Machine Learning for improving model performance and prediction accuracy. Raw healthcare datasets may contain inconsistent, incomplete, or unformatted data, which can affect prediction results. Therefore, preprocessing techniques were applied before training the Machine Learning models.

The following preprocessing steps were performed in the proposed system.

Data Cleaning

Data cleaning was performed to improve dataset quality and remove inconsistencies. The following operations were carried out:

- Removed inconsistent records
- Handled missing values
- Standardized symptom names

This step ensured that the dataset was clean and suitable for training Machine Learning algorithms.

Data Preparation

The dataset was formatted and prepared for:

- Multi-class disease classification
- Symptom-based disease prediction

Proper data preparation ensured efficient training, accurate prediction, and better overall performance of the Machine Learning models.

4.6 Machine Learning Algorithms

To identify the best-performing model for disease prediction, multiple Machine Learning algorithms were trained and evaluated using the prepared symptom-based dataset. The algorithms were selected based on their suitability for classification problems and healthcare prediction systems.

The following Machine Learning algorithms were used in the proposed system:

1. Decision Tree
2. Random Forest
3. Support Vector Machine (SVM)
4. Naïve Bayes

4.6.1 Decision Tree

Decision Tree is a supervised Machine Learning algorithm used for classification and decision-making tasks. It creates a tree-like structure where each node represents a condition based on input features.

In the proposed system, the Decision Tree algorithm was used to classify diseases based on symptoms selected by the patient.

Advantages

- Easy to understand and interpret
- Fast prediction process
- Suitable for classification problems

Limitations

- Overfitting on small datasets
- Sensitive to small variations in data
- Lower stability compared to ensemble methods

4.6.2 Random Forest

Random Forest is an ensemble Machine Learning algorithm that combines multiple decision trees to improve prediction accuracy and stability.

The algorithm generates multiple trees and produces the final prediction based on majority voting.

Advantages

- High prediction accuracy
- Better stability and reliability
- Reduces overfitting problems

Limitations

- Higher computational complexity
- Requires larger datasets
- Slower training compared to simple models

4.6.3 Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and pattern recognition tasks. It works by finding an optimal boundary that separates different classes.

In the proposed system, SVM was used for disease classification based on symptom patterns.

Advantages

- Effective for classification tasks
- Works well with high-dimensional data
- Good prediction performance

Limitations

- Computationally expensive
- Complex implementation
- Less efficient for large datasets

4.6.4 Naive Bayes

Naïve Bayes is a probabilistic Machine Learning algorithm based on Bayes' theorem. It assumes that all features are independent of each other.

The algorithm is highly suitable for symptom-based disease prediction because the dataset mainly contains categorical symptom features.

Advantages

- Fast and efficient
- Suitable for categorical symptom data
- Requires less computational power
- Simple implementation
- High prediction performance

Limitations

- Assumes feature independence
- Prediction quality depends on dataset quality

The Naïve Bayes algorithm was found to be the most suitable model for the proposed healthcare dataset due to its high accuracy, low variance, and efficient handling of categorical medical data.

4.7 Model Evaluation

The Machine Learning models used in the proposed system were evaluated using different performance metrics to determine the most suitable algorithm for disease prediction. The evaluation process helped in comparing prediction accuracy, stability, and overall reliability of the models.

The following Machine Learning algorithms were evaluated:

- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- Naïve Bayes

Evaluation Metrics

The following performance metrics were used for model evaluation.

Accuracy

Accuracy measures the percentage of correctly predicted disease records out of the total predictions made by the model. Higher accuracy indicates better prediction performance.

Stability

Stability measures the consistency and reliability of prediction results across multiple runs. Lower variance indicates better model stability.

Standard Deviation

Standard deviation measures the variation in model performance. A lower standard deviation means the model produces more stable and consistent results.

Model Performance Comparison

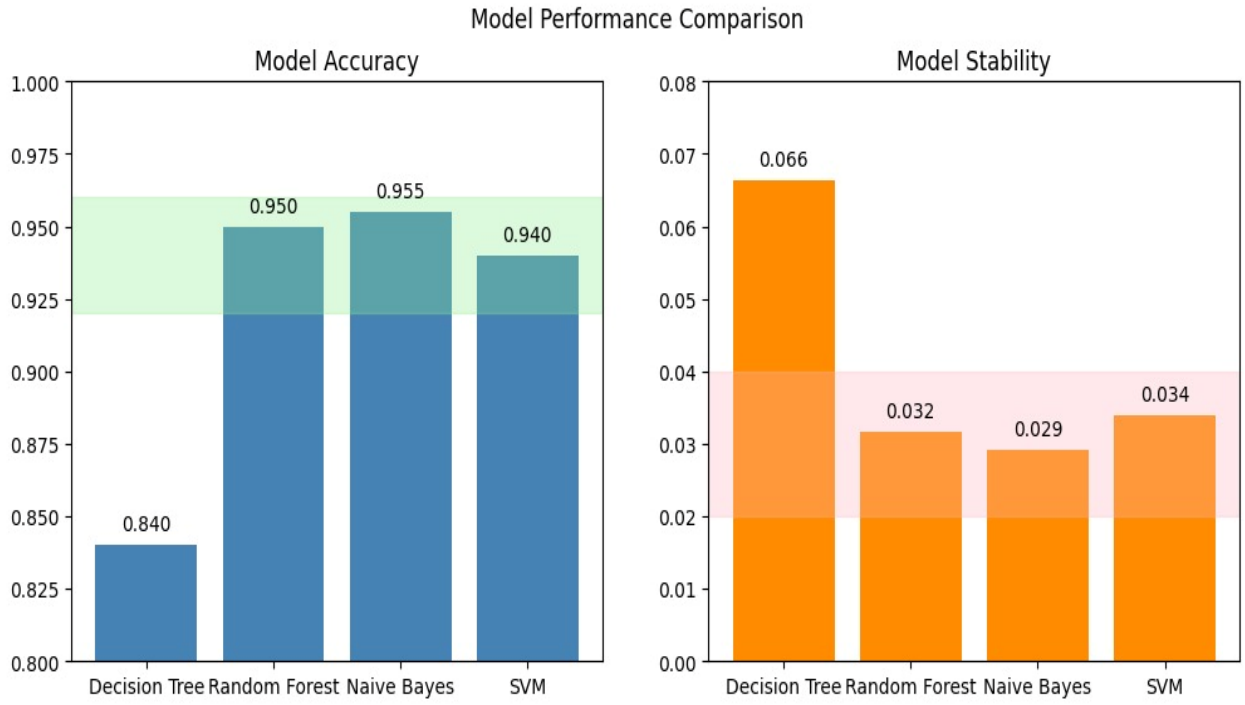
The performance comparison of all Machine Learning algorithms is shown in Table 4.2.

Table 4.2: Performance Comparison of Machine Learning Models

Model	Accuracy	Stability (Std Dev)	Reason for Lower Performance
Decision Tree	84.0%	0.066	Overfitting on small datasets and sensitivity to small data variations reduced reliability.
Random Forest	95.0%	0.032	More complex model requiring larger datasets for optimal learning.
SVM	94.0%	0.034	Performs better on high-dimensional numerical data rather than categorical symptom data.
Naïve Bayes	95.5%	0.029	Best performing model with highest accuracy and lowest variance.

Graphical Analysis

The graphical representation of model performance comparison is shown in Figures



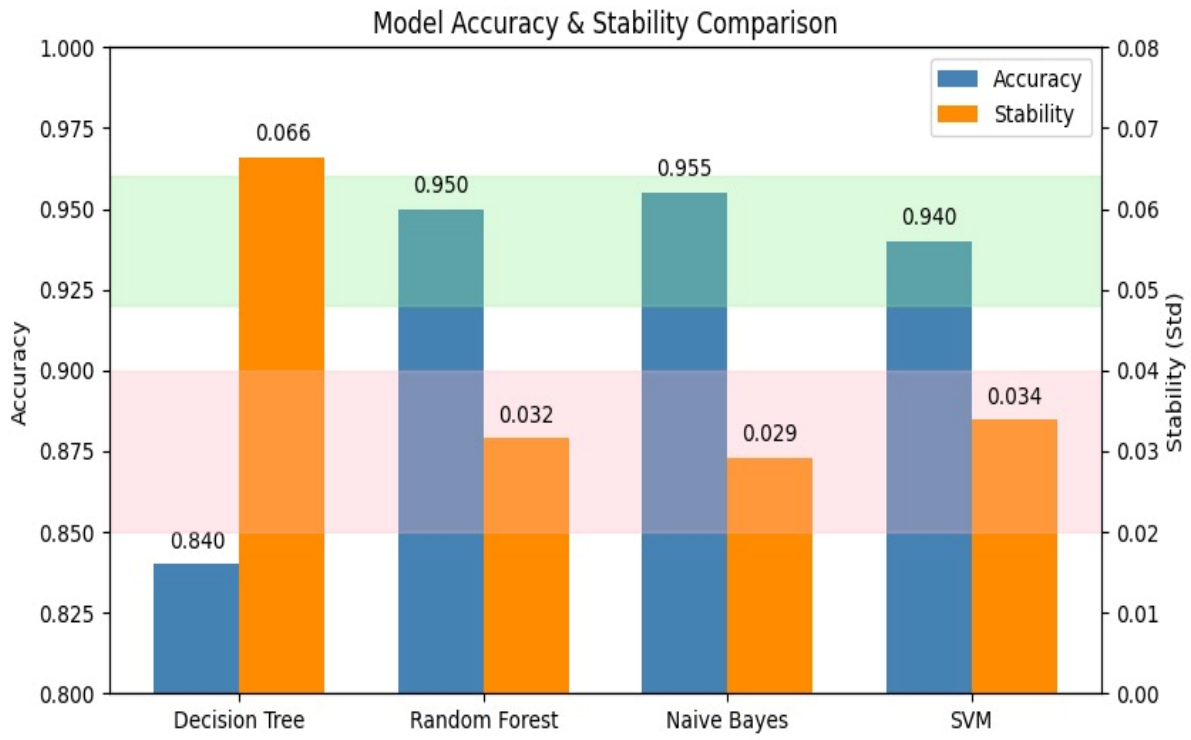


Figure 4.3: Model Performance Comparison using Accuracy and Stability Metrics

The graph compares the performance of all Machine Learning models based on:

- Prediction accuracy
- Stability (standard deviation)

From the graph, it can be observed that:

- Decision Tree achieved the lowest accuracy and highest variance due to overfitting.
- Random Forest provided high accuracy but required more computational complexity.
- SVM achieved good accuracy but was less suitable for categorical symptom-based data.
- Naïve Bayes achieved the highest accuracy (95.5%) and lowest variance (0.029), making it the most reliable model for the proposed system.

The evaluation results clearly indicate that the Naïve Bayes algorithm is the most suitable Machine Learning model for symptom-based disease prediction in the proposed healthcare system.

4.8 Final Model Selection

After comparing all the Machine Learning algorithms, the Naïve Bayes algorithm was selected as the final prediction model for the proposed Clinic Management System.

The selection was based on performance evaluation metrics such as accuracy, stability, prediction speed, and suitability for symptom-based healthcare data. Among all the evaluated algorithms, Naïve Bayes achieved the best overall performance and provided more reliable prediction results.

Reasons for Selecting Naïve Bayes

The major reasons for selecting the Naïve Bayes algorithm are as follows:

- Highest prediction accuracy (95.5%)
- Lowest variance and better stability (0.029)
- Fast prediction performance
- Efficient handling of categorical symptom data
- Simple and lightweight implementation
- Requires less computational resources
- Suitable for multi-class disease classification

Compared to other algorithms, Naïve Bayes performed better because the dataset mainly consisted of categorical symptom features, which are efficiently handled by probabilistic classification methods.

The model also provided consistent prediction results with lower standard deviation, making it more stable and reliable for healthcare applications.

Therefore, the Naïve Bayes algorithm was selected as the final Machine Learning model for disease prediction in the proposed system. The selected model provides accurate disease prediction and supports efficient doctor recommendation and healthcare management services.

CHAPTER 5

IMPLEMENTATION DETAILS

5.1 Software Requirements

The proposed system requires both hardware and software components for successful development and execution. The following requirements were used during the implementation of the project.

Hardware Requirements

The minimum hardware requirements for the system are:

Component	Requirement
Processor	Intel i5 or above
RAM	8 GB
Storage	256 GB
Internet Connection	Required
Operating System	Windows/Linux

The hardware configuration provides sufficient resources for running the web application, database server, and Machine Learning model efficiently.

Software Requirements

The following software technologies and tools were used for system development:

Software	Purpose
Java 17	Backend programming language
Spring Boot	Backend framework
MySQL	Relational database
Maven	Dependency management
VS Code / IntelliJ IDEA	Development environment
Browser	Running the web application

These technologies provide scalability, security, and efficient system performance.

5.2 Frontend Implementation

The frontend of the proposed system is developed to provide an interactive and user-friendly interface for patients, hospitals, and administrators.

The frontend technologies used are:

- HTML
- CSS
- Bootstrap
- Thymeleaf

HTML

HTML is used for designing the structure of web pages and forms.

CSS

CSS is used for styling and improving the appearance of the user interface.

Bootstrap

Bootstrap is used to create responsive and mobile-friendly web pages.

Thymeleaf

Thymeleaf is integrated with Spring Boot for dynamic page rendering and server-side template management.

Features of Frontend

The frontend provides several important features such as:

- Responsive user interface
- User dashboard
- Disease prediction interface
- Appointment booking page
- Login and registration pages
- Doctor management pages
- Admin dashboard

The frontend ensures smooth interaction between users and the backend system.

5.3 Backend Implementation

The backend of the system is implemented using the Spring Boot framework. It handles business logic, data processing, user authentication, and communication with the database.

The application follows the MVC (Model-View-Controller) architecture.

Controller Layer

The Controller layer handles HTTP requests and responses. It acts as an interface between frontend and backend services.

Functions:

- Receives user requests
- Sends responses to frontend
- Manages API endpoints

Service Layer

The Service layer contains business logic and processing functionalities.

Functions:

- Disease prediction processing
- Appointment management
- User validation
- Doctor management

Repository Layer

The Repository layer performs database operations using Spring Data JPA and Hibernate.

Functions:

- CRUD operations
- Database queries
- Data retrieval and storage

Spring Security Integration

Spring Security is integrated into the backend for:

- Authentication
- Authorization
- Role-based access control
- Secure session management

The backend ensures secure and efficient handling of all healthcare operations.

5.4 Database Design

The proposed Clinic Management System with Disease Prediction and Appointment Booking uses a MySQL relational database for storing and managing healthcare-related information efficiently. The database is designed using relational modeling techniques to maintain data consistency, integrity, and efficient data retrieval.

The Entity Relationship (ER) diagram of the database was generated using the Reverse Engineering feature available in MySQL Workbench. After creating the database schema and relationships, reverse engineering was performed to automatically generate the ER diagram representing the structure of the system database.

The database stores information related to:

- Users
- Doctors
- Appointments
- Time Slots
- Diseases
- Symptoms

The database design supports efficient appointment management, disease prediction, and role-based healthcare operations.

Main Tables

1. Users Table

The users table stores authentication and user-related information.

Attributes:

- id
- username
- password
- role

Purpose:

- Stores login credentials
- Maintains role-based access control
- Supports authentication and authorization

2. Doctor Table

The doctor table stores doctor-related information.

Attributes:

- id

- name
- specialization

Purpose:

- Stores doctor details
- Maintains specialization information
- Supports doctor recommendation functionality

3. Appointment Table

The appointment table stores appointment booking details.

Attributes:

- id
- date
- doctor_id
- patient_id
- timeslot_id

Purpose:

- Stores appointment records
- Maps patients with doctors
- Maintains booking information

4. Time Slot Table

The time_slot table stores doctor availability and slot booking information.

Attributes:

- id
- start_time
- end_time
- booked
- doctor_id

Purpose:

- Maintains doctor availability
- Prevents slot conflicts
- Supports appointment scheduling

5. Disease Table

The disease table stores disease-related information.

Attributes:

- id
- name
- required_specialization

Purpose:

- Stores disease names
- Maintains doctor specialization mapping
- Supports disease prediction output

6. Symptom Table

The symptom table stores symptom information.

Attributes:

- id
- name

Purpose:

- Stores symptom names
- Used in disease prediction processing

7. Disease_Symptom Table

The disease_symptom table acts as a junction table between diseases and symptoms.

Attributes:

- disease_id
- symptom_id

Purpose:

- Implements many-to-many relationship
- Maps diseases with multiple symptoms
- Supports symptom-based prediction

Relationships Between Tables

One-to-Many Relationships

Doctor → Time Slot

One doctor can have multiple available time slots.

User → Appointment

One patient can book multiple appointments.

Doctor → Appointment

One doctor can handle multiple appointments.

Many-to-Many Relationship

Disease ↔ Symptom

One disease can contain multiple symptoms, and one symptom can belong to multiple diseases.

This relationship is implemented using the disease_symptom junction table.

ER Diagram

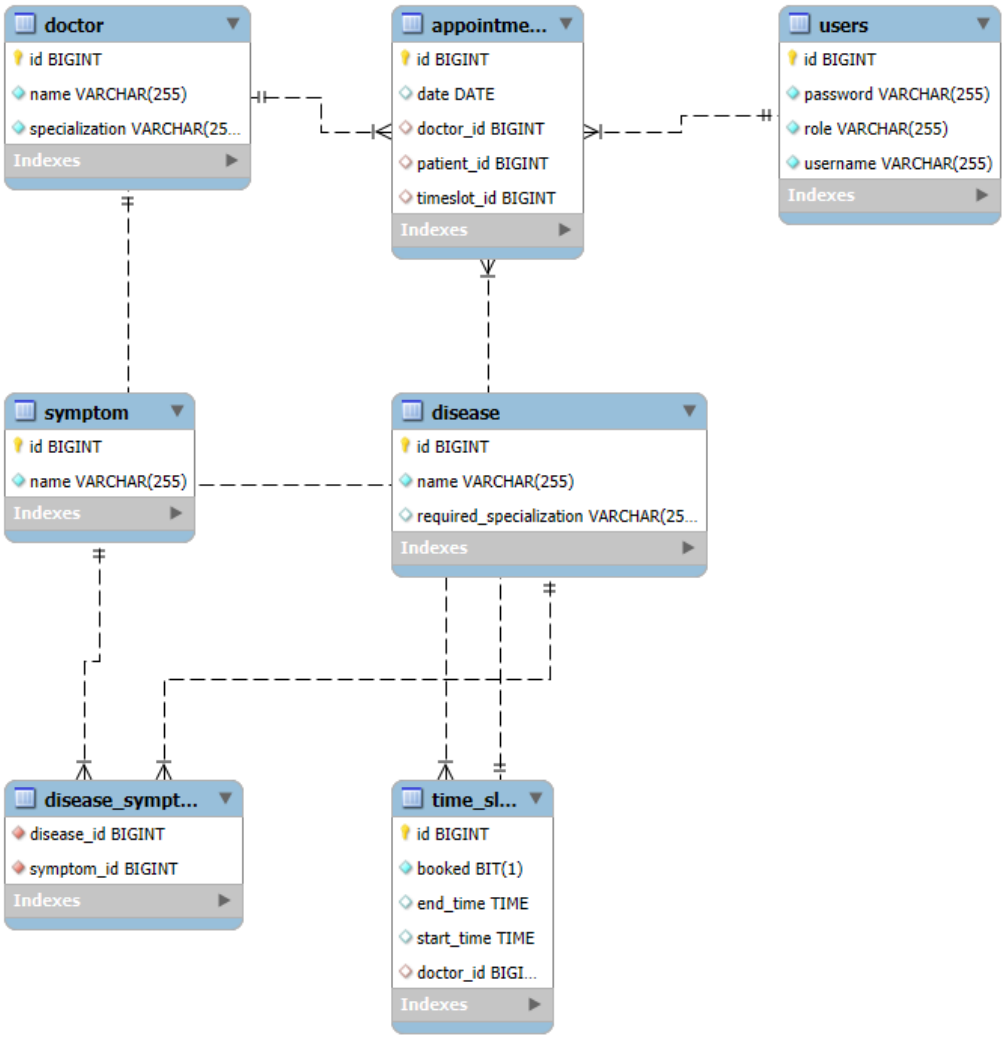


Figure 5.1: Entity Relationship Diagram of Clinic Management System

The database design ensures:

- Proper normalization
- Reduced data redundancy
- Efficient relationship management
- Secure healthcare data storage
- Scalability and maintainability of the system

5.5 Security Implementation

Security is an important part of the proposed healthcare system. The system uses Spring Security to provide secure authentication and authorization mechanisms.

Features of Security Implementation

User Authentication

Users are required to log in using valid credentials before accessing the system.

Password Encryption

Passwords are securely encrypted before storing them in the database.

Role-Based Authorization

Different access permissions are provided for:

- Patient
- Hospital
- Admin

Secure Session Management

User sessions are securely managed to prevent unauthorized access.

The security implementation helps protect sensitive healthcare information and ensures secure system access.

5.6 Technologies Used

The following technologies were used during development of the proposed system:

Technology	Purpose
Spring Boot	Backend Development
MySQL	Database Management
Hibernate	Object Relational Mapping (ORM)
JPA	Database Access Layer
Bootstrap	Frontend Design
Thymeleaf	Dynamic Web Page Rendering
HTML/CSS	User Interface Design
Java 17	Backend Programming
Naïve Bayes	Disease Prediction
Spring Security	Authentication and Authorization

These technologies together provide scalability, flexibility, security, and efficient healthcare management.

CHAPTER 6

RESULTS AND ANALYSIS

6.1 Model Performance Comparison

To identify the most suitable Machine Learning model for disease prediction, multiple classification algorithms were trained and evaluated using the prepared symptom-based dataset. The models were compared based on the following evaluation parameters:

- Accuracy
- Stability
- Standard Deviation

The following Machine Learning algorithms were tested:

1. Decision Tree
2. Random Forest
3. Support Vector Machine (SVM)
4. Naïve Bayes

The performance comparison of the models is shown below:

Model	**Accuracy**	**Stability (Std Dev)**
Decision Tree	84.0%	0.066
Random Forest	95.0%	0.032
Support Vector Machine (SVM)	94.0%	0.034
Naïve Bayes	95.5%	0.029

From the comparison results, it was observed that the Naïve Bayes algorithm achieved the highest prediction accuracy and lowest variance among all tested models.

Reasons for Better Performance of Naïve Bayes

- Efficient handling of categorical symptom data
- Fast prediction speed
- Low computational complexity

- Better stability compared to other models
- Suitable for multi-class disease classification

Therefore, Naïve Bayes was selected as the final disease prediction model for the proposed system.

6.2 Accuracy and Stability Analysis

The selected Naïve Bayes model was further analyzed using performance evaluation metrics.

Accuracy Analysis

The Naïve Bayes model achieved:

- **Accuracy: 95.5%**

The high accuracy indicates that the model can correctly predict diseases for most symptom combinations provided by users.

Stability Analysis

The stability of the model was measured using standard deviation.

The model achieved:

- **Stability (Standard Deviation): 0.029**

A lower standard deviation indicates that the model produces consistent and reliable prediction results.

Performance Summary

The Naïve Bayes model provided:

- High prediction accuracy
- Stable performance
- Faster execution
- Reliable disease classification

The evaluation results confirm that the model is suitable for symptom-based disease prediction applications.

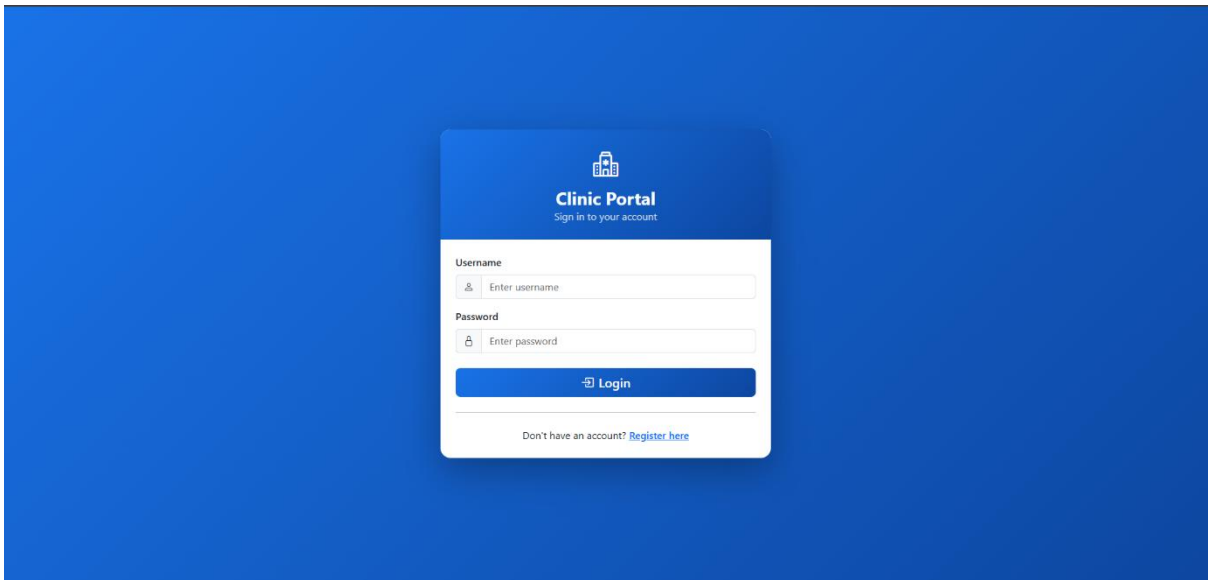
6.3 System User Interface Screens

The proposed system includes multiple user interfaces designed for patients, hospitals, and administrators. The interfaces are developed using HTML, CSS, Bootstrap, and Thymeleaf to provide a responsive and user-friendly experience.

The following system interfaces were implemented:

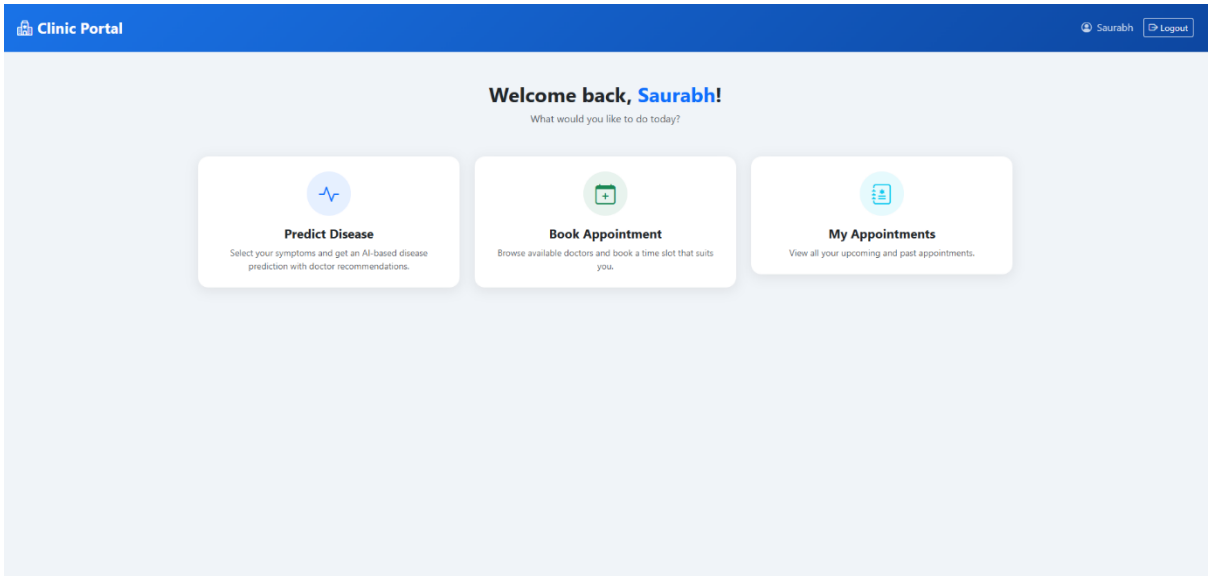
Login Page

The login page allows users to securely access the system using authentication credentials.



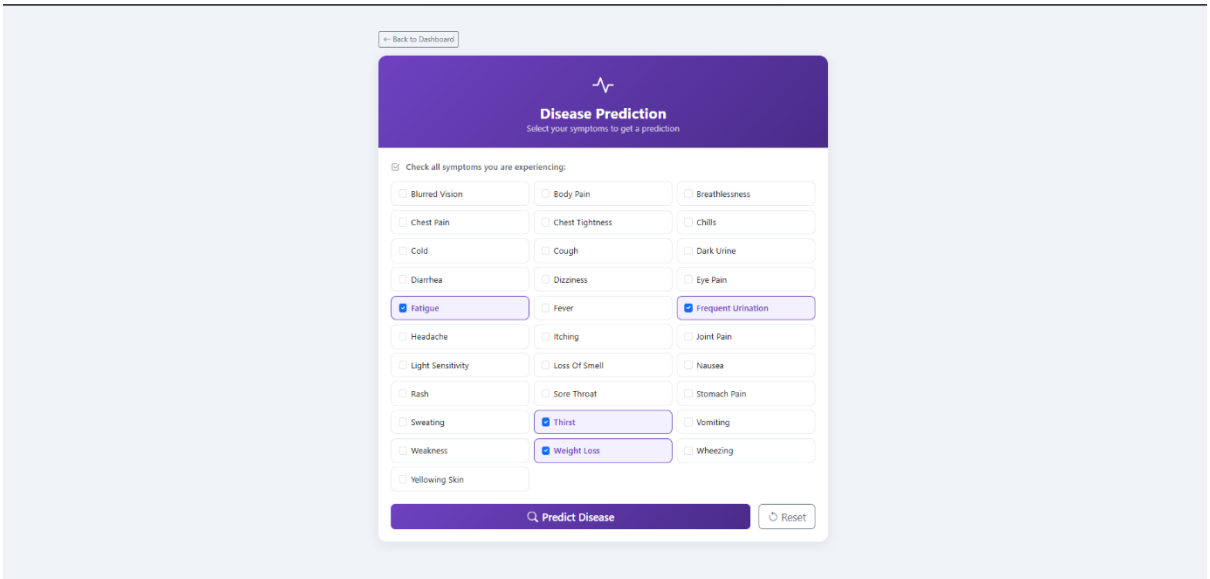
User Dashboard

The user dashboard provides quick access to disease prediction, appointment booking, and appointment history.



Disease Prediction Input

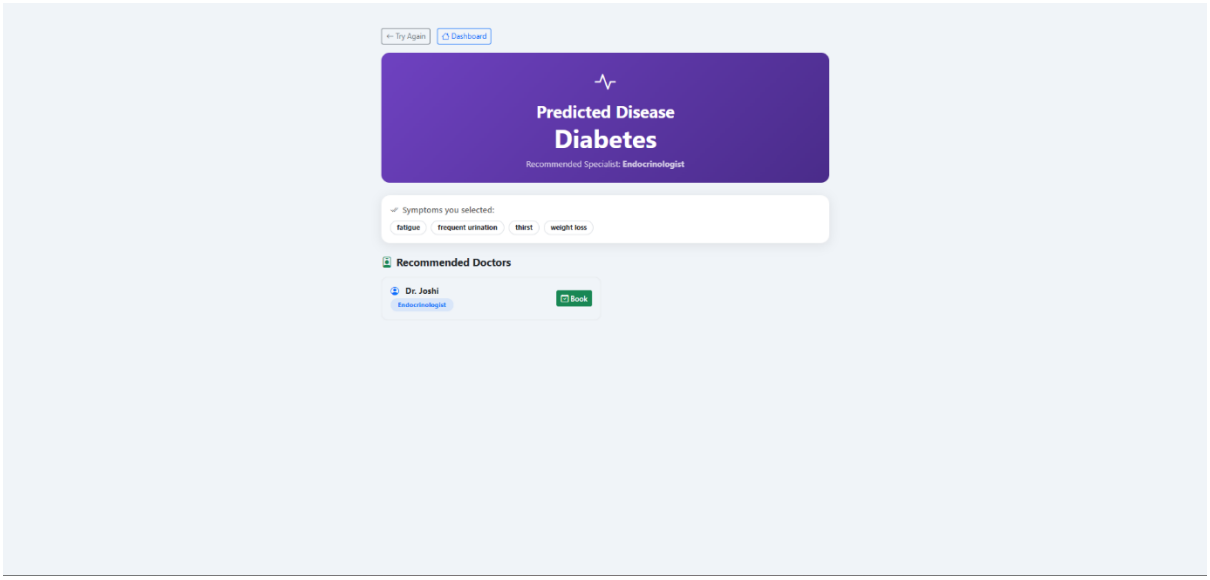
This interface allows patients to select symptoms for disease prediction.



Disease Prediction Output

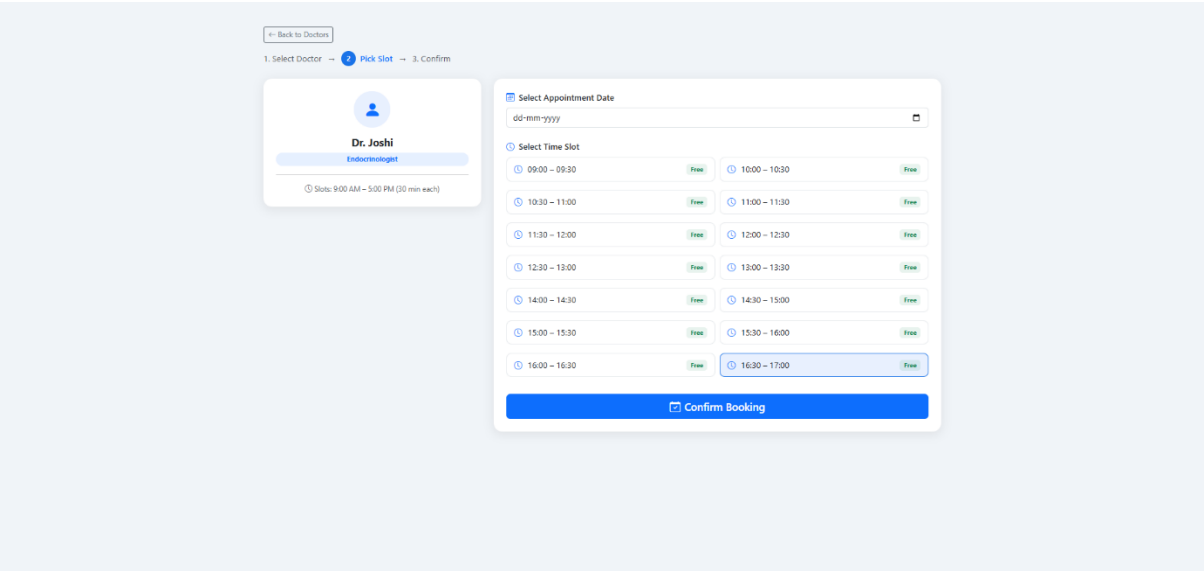
The output page displays:

- Predicted disease
- Recommended doctor specialization
- Additional healthcare recommendations



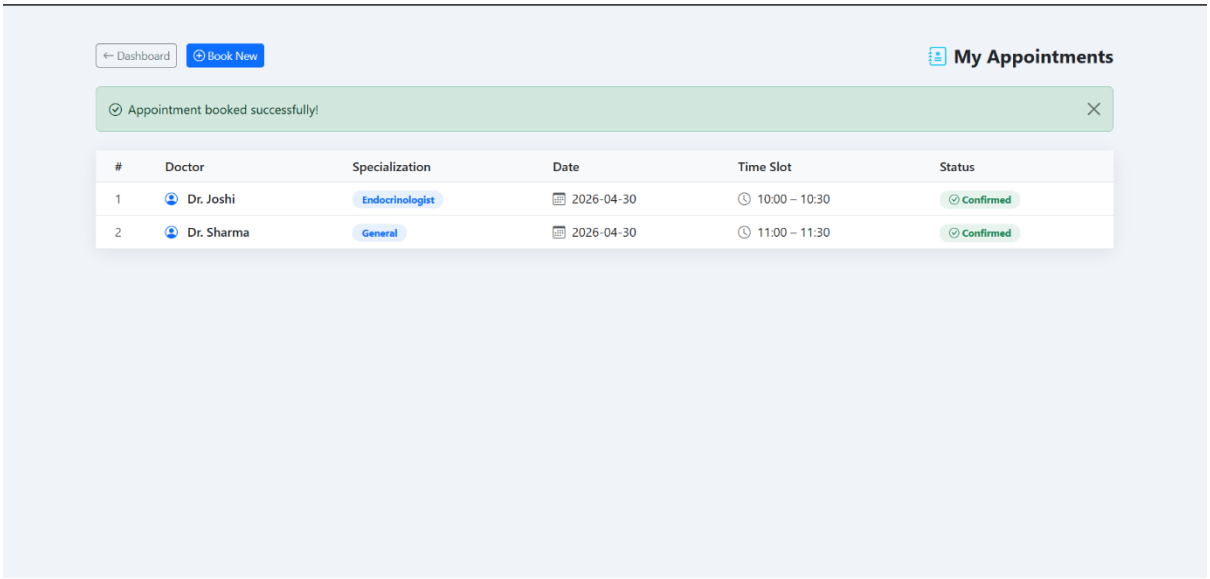
Appointment Booking Page

Patients can select doctors and available time slots for booking appointments.



Appointment Confirmation

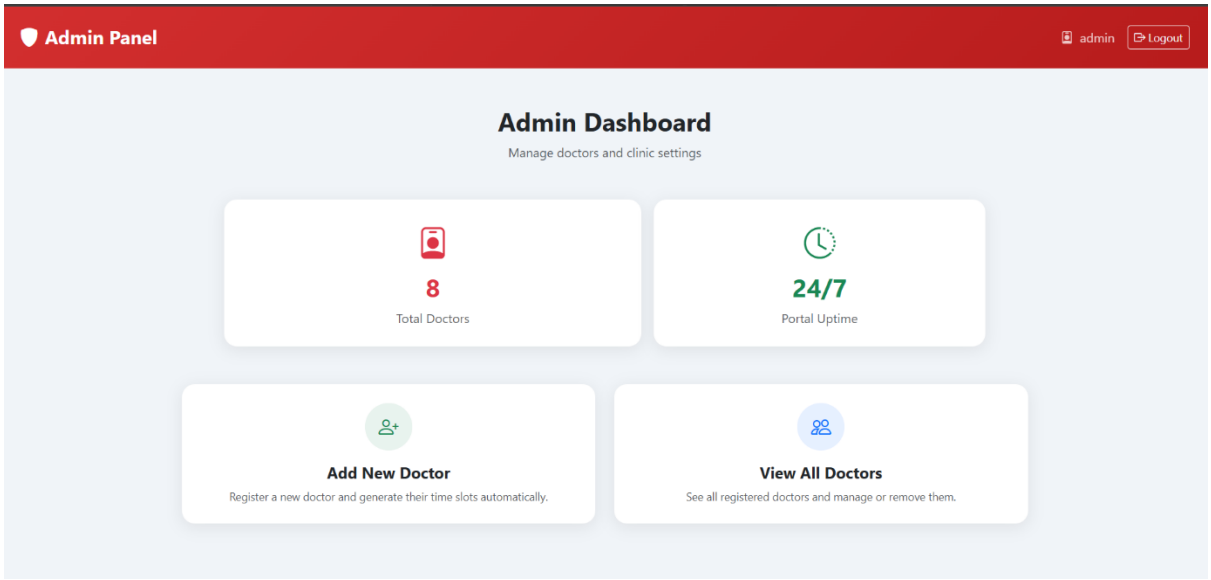
The system confirms successful appointment booking and stores appointment details.



Admin Dashboard

The admin dashboard provides functionalities for:

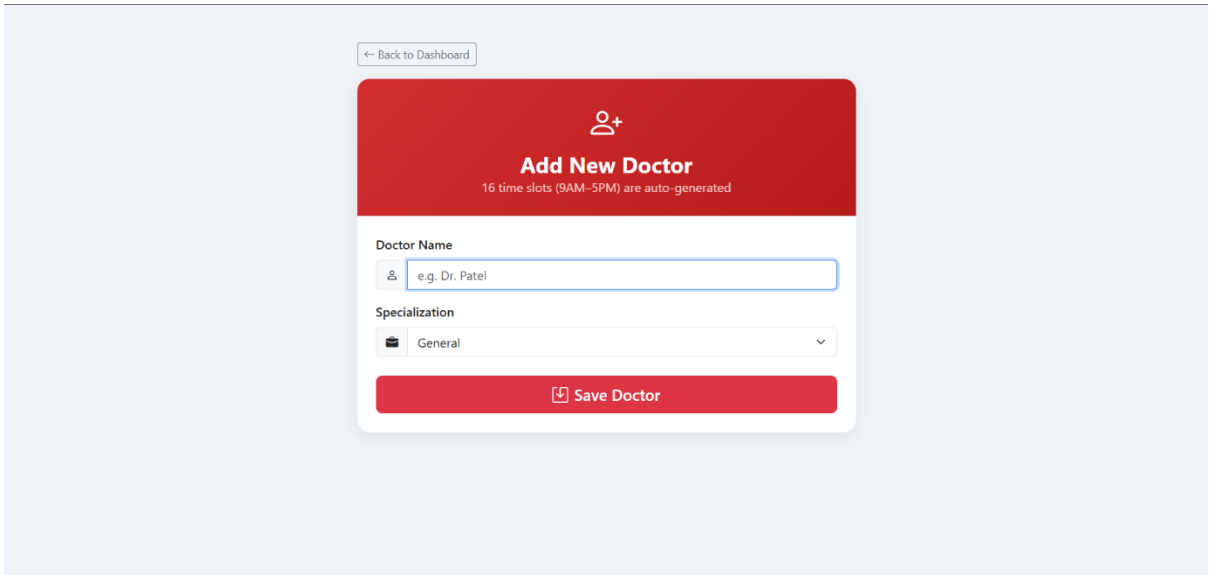
- Doctor management
- Hospital management
- System monitoring



Doctor Management Pages

These interfaces allow hospitals/admins to:

- Add doctors
- Update doctor information
- Manage doctor availability



6.4 Result Analysis

The proposed Clinic Management System successfully integrates healthcare management functionalities with Machine Learning-based disease prediction.

The system effectively provides:

- Disease prediction
- Doctor recommendation
- Online appointment booking
- Slot management
- Role-based access control

Benefits Achieved

1. **Reduced Manual Effort**
   Automation of appointment booking and record management significantly reduces manual work.
2. **Faster Appointment Booking**
   Patients can quickly book appointments without visiting clinics physically.
3. **Better Patient Experience**
   The integrated healthcare platform improves user convenience and accessibility.
4. **Improved Clinic Management**
   Hospitals can efficiently manage doctors, slots, and appointments using the system.
5. **Intelligent Disease Prediction**
   The Machine Learning model provides preliminary disease prediction support for patients.

Overall, the proposed system delivers a smart, secure, efficient, and user-friendly healthcare management solution.

CHAPTER 7

DEPLOYMENT AND CLOUD HOSTING

6.1 Need for Deployment

Software development does not end after successful implementation and testing. For real-world usage, an application must be accessible to end users through the internet. Deployment is the process of hosting an application on a server so that users can access it from anywhere using a web browser.

In the Clinic Management System, deployment was required to:

• Make the application accessible online.
• Allow users to access the system from any device.
• Demonstrate real-world software engineering practices.
• Enable cloud-based database connectivity.
• Provide a production-ready healthcare solution.

Without deployment, the application would run only on the developer's local machine and would not be available to external users.

6.2 Local Environment vs Cloud Deployment

During development, the Clinic Management System was executed locally using localhost and a local MySQL database.

Local Environment
• Accessible only on developer machine
• Uses localhost
• Local MySQL database
• Suitable for development and testing
• Not accessible over internet

Cloud Deployment
• Accessible from anywhere
• Hosted on cloud server
• Uses cloud database
• Suitable for production use
• Publicly accessible through URL

The transition from local execution to cloud deployment required additional technologies such as Docker, GitHub, Railway MySQL, and Render.

6.3 Available Cloud Deployment Platforms

Several cloud platforms are available for hosting modern web applications.

Platform Features
AWS Enterprise cloud services
Microsoft Azure Cloud hosting and database services
Google Cloud Platform Scalable cloud infrastructure
Railway Cloud database and application hosting
Render Easy deployment with Docker support
Heroku Platform-as-a-Service hosting

Each platform provides different deployment capabilities depending on project requirements and budget.

6.4 Platform Selection

For this project, Render was selected as the application hosting platform and Railway was selected as the cloud database provider.

Reasons for selecting Render:

- Free hosting tier available.
- Direct GitHub integration.
- Docker support.
- Automatic deployment.
- Easy configuration.
- Public URL generation.

Reasons for selecting Railway:

- Free MySQL database service.
- Cloud database management.
- Public database endpoint.
- Simple integration with Spring Boot applications.

This combination provided a complete cloud deployment solution without additional infrastructure costs.

6.5 Project Development

The Clinic Management System was developed locally using IntelliJ IDEA.

Technology stack used:

- Java 17
- Spring Boot
- Spring MVC
- Spring Data JPA
- Hibernate

• Thymeleaf
• Maven
• MySQL

Initially, the application was executed on localhost and connected to a local MySQL database.

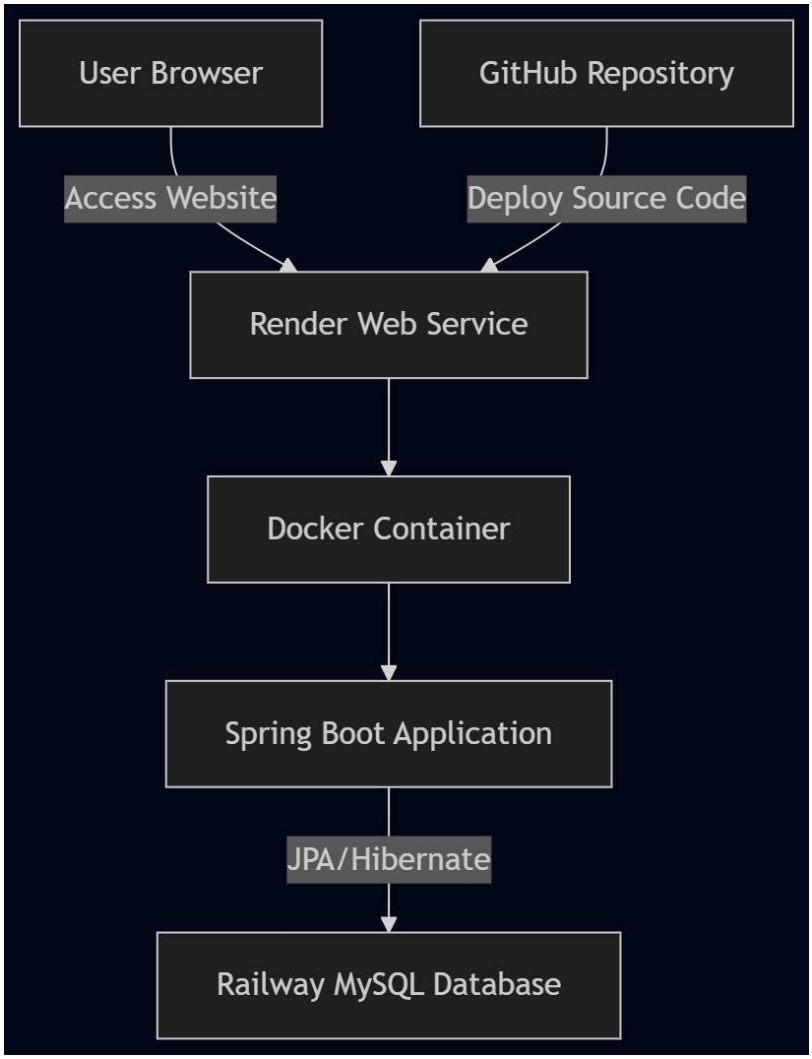


Figure 7.1: Cloud Deployment Architecture

6.6 Local Database Setup

Initially, application.properties contained local database credentials.

Example:

spring.datasource.url=jdbc:mysql://localhost:3306/clinic_db
spring.datasource.username=root
spring.datasource.password=*****

This configuration worked only on the developer's machine.

Since cloud-hosted applications cannot access local databases, a cloud-hosted MySQL database was required.

6.7 Environment Variable Configuration

To support both local and cloud environments, environment variables were introduced.

Configuration:

spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/clinic_db}

spring.datasource.username=${DB_USERNAME:root}

spring.datasource.password=${DB_PASSWORD:*****}

server.port=${PORT:8080}

Locally, default values are used. During deployment, Render provides production values through environment variables.

This approach improves security and portability.

6.8 Maven Build Process

Apache Maven was used for dependency management and project packaging.

Command used:

mvn clean package

The build process:

1. Downloads dependencies.
2. Compiles source code.
3. Runs packaging tasks.
4. Generates executable JAR file.

The generated JAR file is stored inside the target directory and contains the complete Spring Boot application.

6.9 Docker Containerization

Docker was used to package the application into a portable container.

Benefits of Docker:

• Consistent runtime environment.
• Easy deployment.
• Dependency isolation.
• Platform independence.

Instead of manually installing Java and Maven on the server, Docker packages everything into a single container image.

6.10 Git and GitHub Integration

Git was used for version control.

Major commands:

git init
git add .
git commit -m "Initial Commit"
git push

GitHub served as the central repository from which Render automatically fetched source code.

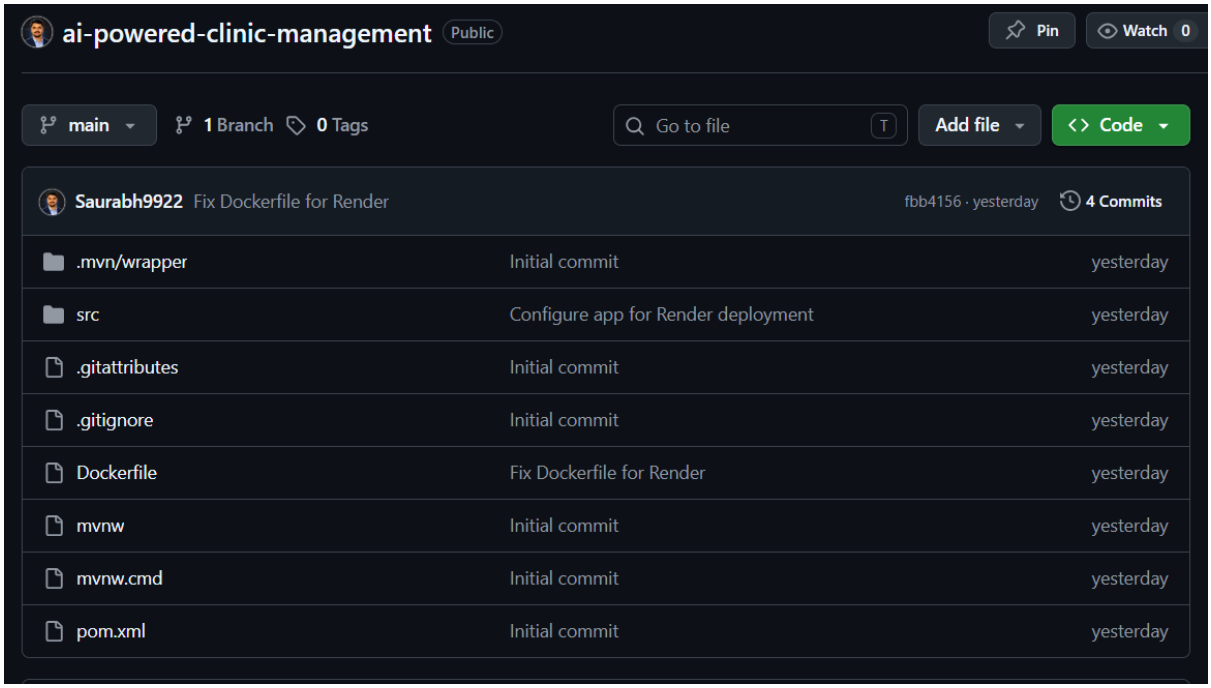


Figure 7.2: GitHub Repository of the Clinic Management System

6.11 Railway MySQL Database

A Railway project was created and a MySQL database instance was provisioned.

Railway generated:

- MYSQLHOST
- MYSQLPORT
- MYSQLDATABASE
- MYSQLUSER
- MYSQLPASSWORD
- MYSQL_PUBLIC_URL

These values were used by the deployed application for database connectivity.

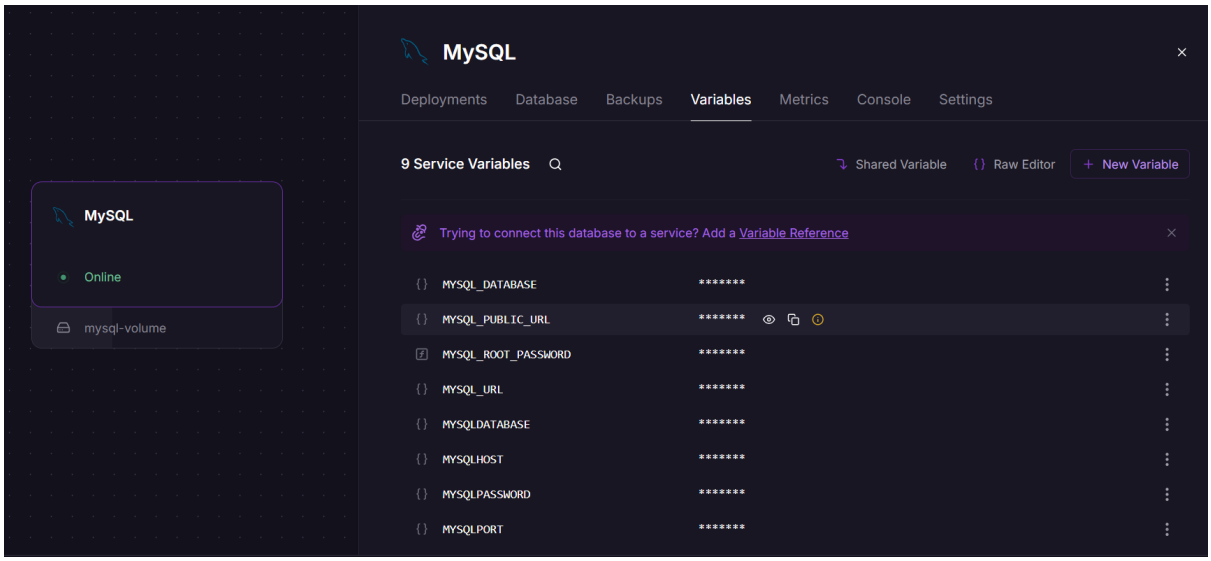


Figure 7.3: Railway MySQL Database Configuration

6.12 Render Deployment

Render was used to host the Spring Boot application.

Deployment process:

1. Connect GitHub repository.
2. Detect Dockerfile.
3. Build Docker image.
4. Start application container.
5. Expose public URL.

Render automatically rebuilds and redeploys the application whenever changes are pushed to GitHub.

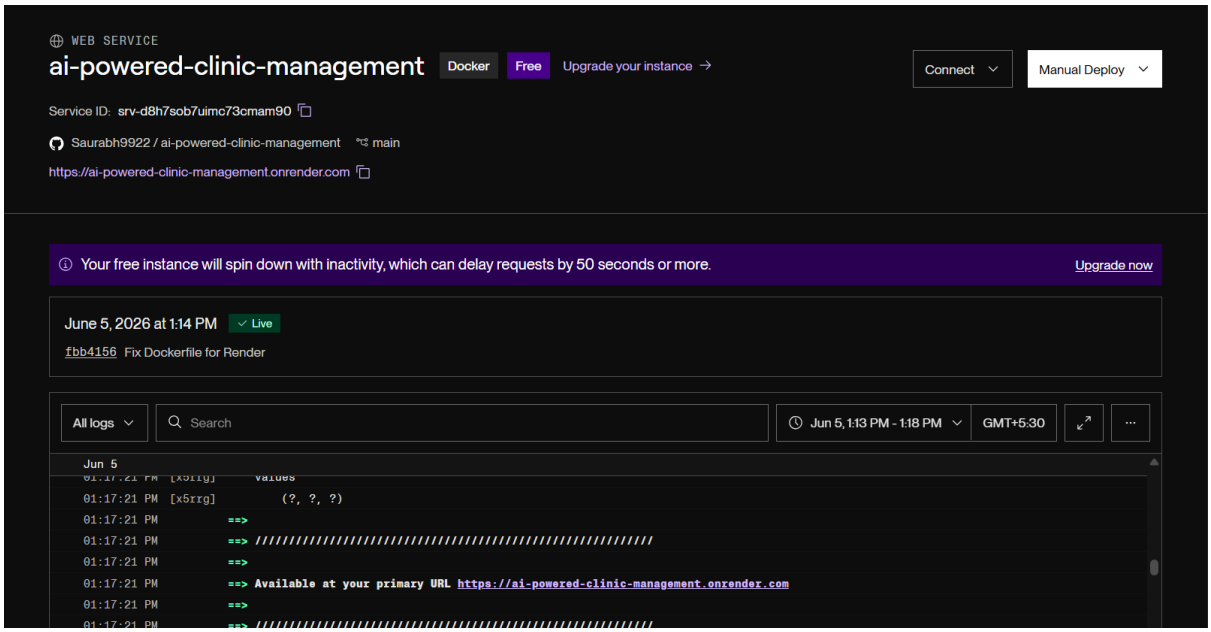


Figure 7.4: Render Deployment Dashboard

6.13 Live Application Access

After successful deployment on the Render cloud platform, the Clinic Management System became publicly accessible through a web browser. The live application can be accessed using the URL generated by Render.

Live Application URL:

https://ai-powered-clinic-management.onrender.com

The hosted application provides all functionalities available in the local version, including:

• User Registration and Login

• Disease Prediction using Naive Bayes Algorithm

• Doctor Recommendation

• Appointment Booking

• Appointment Management

• Admin Dashboard

Users can access the application from any device connected to the internet without requiring any software installation.

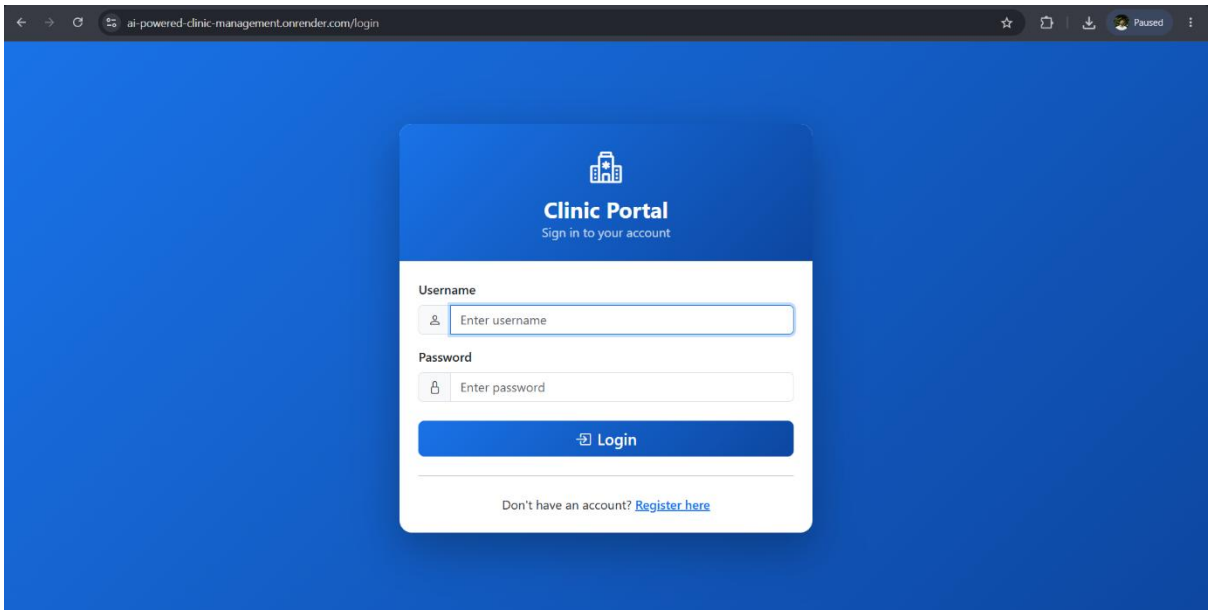


Figure 6.5 live login page of the deployed Clinic Management System hosted on Render.

CHAPTER 7

CONCLUSION

The proposed AI-Powered Clinic Management System successfully integrates Machine Learning and modern web technologies to improve healthcare management services. The system provides intelligent disease prediction, doctor recommendation, online appointment booking, and efficient slot management through a secure and user-friendly platform.

The Naïve Bayes algorithm achieved an accuracy of 95.5%, making it suitable for symptom-based disease prediction and healthcare recommendation. The project successfully achieved its objectives, including automation of clinic operations, intelligent disease prediction, appointment management, role-based access control, and improved healthcare accessibility.

To make the system production-ready, the application was containerized using Docker and deployed on the Render cloud platform with Railway MySQL as the cloud-hosted database. The use of GitHub for version control and cloud deployment demonstrates the practical implementation of modern software engineering and cloud computing concepts.

Overall, the developed system provides a smart, efficient, scalable, and reliable healthcare solution for clinics and hospitals. Future enhancements such as telemedicine integration, online payment systems, mobile application support, and real-time healthcare datasets can further improve the capabilities of the proposed system.

CHAPTER 8

FUTURE SCOPE

---

1. Real-Time Clinical Dataset Integration

The system can be enhanced by integrating real-time patient datasets collected from hospitals and healthcare organizations. Training the Machine Learning model using real clinical data can improve disease prediction accuracy, reliability, and overall healthcare recommendations.

2. Online Payment Gateway Integration

The system can be extended by integrating secure online payment gateways for appointment booking and healthcare services. This feature will enable patients to make digital payments, manage billing records, and reduce manual payment handling.

3. Telemedicine and Video Consultation

Future versions of the system can include telemedicine support with video consultation features. Patients will be able to consult doctors remotely, receive online prescriptions, and access healthcare services without physically visiting hospitals or clinics.

4. Mobile Application Development

A mobile application version of the system can be developed for Android and iOS platforms. The mobile application will provide easier accessibility, appointment notifications, disease prediction support, and healthcare services anytime and anywhere.

CHAPTER 9

REFERENCES

---

[1] A. Satija, P. Pahuja, D. Singh, and A. Hussain, *Prediction in Medicine: The Impact of Machine Learning on Healthcare*, Elsevier, Oct. 2024.

[2] P. Jyothi, A. Lokesh Kumar, and D. Dakshayani, "Disease Prediction Using Naïve Bayes, Random Forest, Decision Tree and KNN Algorithms," *i-Manager's Journal on Computer Science*, vol. 11, no. 4, pp. 12–20, Jan. 2024.

[3] R. Shukla, "Disease Prediction System Using Support Vector Machine and Random Forest Classifiers," *International Journal of Innovative Science and Research Technology*, Mar. 2023.

[4] *ClinicCare Manager: An Efficient Electronic Health Record (EHR) and Healthcare Management System Using Spring Boot and React*, Jan. 2026.

[5] T. Mendes, "Integration of IoT Health Systems and Hospital Management Using Spring Boot and API Frameworks," *Proceedings of the International Conference on Artificial Intelligence and Engineering*, 2025.

[6] S. Hossain, M. Rahman, and T. Islam, "Machine Learning Approaches for Multi-Disease Prediction Using Ensemble Techniques," *IEEE Access*, vol. 12, pp. 45890–45905, 2024.

[7] Y. Chen, L. Zhang, and H. Li, "Comparative Analysis of Supervised Learning Algorithms for Healthcare Diagnosis," *IEEE Transactions on Computational Biology and Bioinformatics*, vol. 21, no. 3, pp. 1456–1468, 2024.

[8] M. K. Sharma and R. Gupta, "Design and Implementation of a Secure Hospital Management System Using Spring Boot and Microservices," in *Proceedings of IEEE International Conference on Computing, Communication and Intelligent Systems (ICCCIS)*, 2023.

[9] A. Verma and S. Kulkarni, "Web-Based Smart Healthcare Management System Using Machine Learning Techniques," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 2, pp. 210–218, 2025.

[10] J. Lee and K. Park, "Artificial Intelligence-Based Disease Prediction and Online Appointment System for Smart Healthcare," *Journal of Healthcare Engineering*, vol. 2024, pp. 1–11, 2024.